ClusterSSFDA: Clustered Semi-Supervised Federated Domain Adaptation

Michele Craighero*, Taguhi Mesropyan*, Diego Carrera†, Beatrice Rossi†,
Diego Stucchi†, Pasqualina Fragneto† and Giacomo Boracchi*

*Politecnico di Milano, Milano (MI) 20133, Italy
{name.surname}@polimi.it

†STMicroelectronics, Agrate Brianza (MB) 20864, Italy
{name.surname}@st.com

2025 IEEE International Conference on Data Mining (ICDM) | 979-8-3315-9599-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDM65498.2025.00120

***Abstract*—Most Federated Learning (FL) approaches assume a single global model is updated locally by clients and aggregated at the server. However, the *single-model* assumption is often too restrictive, especially in scenarios involving a large amount of different users, where adaptation to different domains and personalization are necessary to improve model performance. In this paper, we propose ClusterSSFDA, the first FL framework to leverage clustering for addressing domain shifts in Semi-Supervised Federated Learning (SSFL), where clients collect data without supervision. In particular, ClusterSSFDA clusters clients based on their models' agreement, and updates multiple models at the server, each tailored to a cluster of similar clients. ClusterSSFDA significantly improves adaptation in SSFL, striking a balance between a single global model, which may be suboptimal in the presence of domain shift among the clients, and fully personalized models, which would be trained on too small datasets. Our experiments on real-world scenarios with multiple levels of domain shift demonstrate that ClusterSSFDA outperforms existing methods, achieving superior performance in challenging SSFL settings.**

I. Introduction

Federated Learning (FL) is a distributed learning paradigm where multiple client models, trained locally on private data, are aggregated into a global model at the server and redistributed back to the clients iteratively over several communication rounds. Most FL methods are inherently *supervised*, requiring labeled data on clients. However, obtaining such annotations is often impractical due to uncooperative or inaccurate users or to the lack of suitable labeling interfaces. *Semi-Supervised Federated Learning* (SSFL) relaxes this requirement and builds the distributed learning scheme assuming partially labeled data. In particular, we focus on a very realistic scenario within SSFL —referred to as *labels-at-server* SSFL— where annotated data are exclusively available at the server, while clients continuously collect only unlabeled data.

The labels-at-server scenario is common in applications like Human Activity Recognition (HAR), where classifiers are trained to identify activities from wearable device traces. HAR is a typical SSFL use case, as it involves potentially an unlimited number of clients for which privacy concerns and communication limits prevent transferring raw data to the server. Moreover, obtaining activity labels on client devices is challenging, as many sensing devices lack suitable user interfaces for labeling. Not surprisingly, HAR datasets are typically created through controlled protocols specifying user activities [1], [2], [3], while training classifiers on public datasets at the server remains relatively straightforward.

In addition to the lack of client annotations, SSFL methods need to face a critical, yet often overlooked, challenge: *domain shift*. Domain shift refers to a discrepancy in data distributions either between the server and clients (*server-client* domain shift) or among the clients themselves (*client-client* domain shift). For example, in applications such as HAR, data heterogeneity arises from user-specific traits and differences in sensing devices, leading to both types of domain shifts. Domain shift is widely recognized as a harm for machine learning models' performance [4]. Despite its importance, the domain shift problem in SSFL has received limited attention.

Within SSFL, the two main approaches are: *i*) *pseudo-labeling* [5], [6] which updates client models via supervised learning on local data using labels predicted by the current model, (i.e., pseudo-labels) and *ii*) *encoder training* [7], [8], which first updates client encoders using unsupervised losses and at the server aggregates these to obtain an updated model to be combined with a shared classification head. Pseudo-labeling has been proven to be effective only in simplified scenarios without domain shift [9], [8], while encoder training methods like SemiFDA [8] show promise in addressing server-client domain shift.

Existing SSFL methods rely on training a single global model to improve accuracy across all clients. This is also the case of SemiFDA, which assumes that all client data originate from a single underlying distribution. While this assumption helps manage server-client domain shifts, it fails to address client-client domain shifts, where client data distributions are heterogeneous. We argue that assuming a single data-generating process at clients is suboptimal for real-world SSFL applications, where client-client domain shift often arise due to factors such as variations in data acquisition methods (e.g., different devices or sensors).

In this paper, we introduce ClusterSSFDA (Clustered Semi-Supervised Federated Domain Adaptation), a novel framework specifically designed to tackle both server-client and client-client domain shifts in the challenging labels-at-server SSFL scenario. Building on SemiFDA [8], ClusterSSFDA introduces a clustering algorithm at the server to group together clients

2374-8486/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDM65498.2025.00120

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

gathering data from similar distributions. Instead of training a single global model, ClusterSSFDA trains and aggregates separate models for each cluster, effectively addressing both server-client and client-client domain shifts.

The core of ClusterSSFDA is a Federated Clustering scheme, which uses the server's labeled data as a shared reference to group clients based on model agreement, comparing their class-conditioned predictions on server data after local updates. Since client data are unlabeled, only local encoders are aggregated, while we use the shared classification head —trained on the server's labeled data— as a common projection for latent representations extracted by different clients' encoders. ClusterSSFDA strikes a balance between a single global model, which may be suboptimal in the presence of domain shift between the clients, and fully personalized models on each client, which are unreliable due to the limited amount of client data. To assess the effectiveness of ClusterSSFDA, we validate our approach through experiments on benchmark datasets designed to simulate different domain shifts across clients. In particular, we consider two different learning tasks: HAR, where domain shifts are intrinsic, and Digit Classification, where domain shifts are induced by modifying or combining different datasets. Experimental results demonstrate that ClusterSSFDA yields consistent improvements, outperforming existing SSFL methods, including SemiFDA. In particular, in scenarios where supervised oracles indicate a margin for improvement, ClusterSSFDA achieves significant performance gains, demonstrating its effectiveness in addressing domain shifts across clients.[1]

To summarize, our contributions are the following:

- We introduce ClusterSSFDA, a novel SSFL solution designed to effectively address multiple domain shifts. To the best of our knowledge, this is the first approach to leverage clustering for handling domain shifts in an SSFL scenario, without requiring any supervision on the clients.
- We demonstrate the flexibility of ClusterSSFDA, whose clustering scheme can be also combined with supervised FL frameworks such as FedAvg, providing benefits to handle domain shift.
- Through extensive experiments, we provide evidence of the benefits of clustering when facing multiple domain shifts in SSFL, and we show that ClusterSSFDA outperforms existing methods in these challenging settings.

The rest of the paper is organized as follows: Section II discusses related work; Section III formalizes the problem setting; Section IV presents our proposed solution; Section V describes the experimental settings and the results; Section VI provides an in-depth discussion on design choices and further analyses; and Section VII concludes the paper.

II. Related Works

Here we review the literature related to ClusterSSFDA. In particular, we discuss SSFL solutions and Clustered FL approaches in Section II-A and II-B, respectively.

[1]The code is available at https://github.com/Michelec1997/ClusterSSFDA.

A. Semi-Supervised Federated Learning

As stated in Section I, this work focuses on the very realistic *labels-at-server* SSFL scenario, where no labeled data are available on the clients. The mainstream SSFL strategy involves generating *pseudo-labels* for client data using a model pretrained on server data. SemiFL [6] represents the state-of-the-art in this field, integrating centralized semi-supervised learning (SSL) techniques such as MixMatch [10] and FixMatch [11] with an alternating training approach and a specific data augmentation procedure. However, as demonstrated by [8], methods that rely on pseudo-labels struggle in realistic SSFL scenarios, characterized by domain shift between the server and clients, potentially resulting in negative learning.

There are few SSFL approaches that enable unsupervised adaptation at the client level, which we refer to as *encoder training*. One such approach is AE-SSFL, introduced in [7], where clients train AutoEncoders (AEs) to reconstruct unlabeled data. After each communication round, the local AE parameters are sent to the server, which combines them using Federated Averaging (FedAvg) [12]. The server then uses the aggregated feature extractor to compute latent features, and updates the classification head on the labeled server data. The idea behind AE-SSFL is to combine both unsupervised learning at the clients and supervised learning at the server, without relying on possibly incorrect pseudo-labels. However, as demonstrated in [13] and confirmed in [8], also AE-SSFL is suboptimal when there is substantial domain shift between the server and client distributions. Semi-Supervised Federated Domain Adaptation (SemiFDA) [8] is a SSFL method specifically designed to address domain shift. After training an initial classifier on the server using labeled data, SemiFDA establishes an unsupervised learning process at clients to train feature extractors. More in detail, SemiFDA adopts a custom loss function to align feature distributions of the clients' encoders with the one of the server by minimizing the distance of the respective covariance matrices. The covariance at server is used as fixed reference. The updated encoders are aggregated at the server using Federated Averaging and sent back for the next iteration, while the classification head remains fixed to maintain aligned features. Although SemiFDA proved to effectively address domain shift between the server and clients, the authors note significant limitations when there is considerable heterogeneity among clients. In fact, aggregating all clients encoders into a single model is suboptimal in such contexts and no SSFL solution addresses the challenging setting where there is client-client domain shift.

We remark that *labels-at-clients* FL solutions, e.g., [14], [15], [16], [17], fall outside the scope of our work, as they assume to have labeled or partially labeled data as clients.

B. Clustered Federated Learning

Several clustering mechanisms have been proposed in FL to group clients with similar data distribution into more homogeneous clusters and train a model for each cluster. However, to the best of our knowledge, no clustering solutions have been proposed to handle domain shift in the SSFL scenario.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

Therefore, here we discuss existing clustering mechanisms for supervised FL, collectively known as *Clustered Federated Learning* (CFL). The first CFL framework has been introduced by [18] and it is based on recursive bi-partitions of clients along the communication rounds. An alternative approach is IFCA [19], which generates random cluster centers and assigns clients to clusters that minimize their loss values. While IFCA achieves significant accuracy improvements, it requires clients to download all cluster models at each round and is highly sensitive to initialization.

In addition, clustering mechanisms based on the similarity of the clients' model parameters have been proposed [20], [21], [22], which we group into the *weights-similarity* clustering category. In [20], an agglomerative hierarchical clustering approach is proposed, which assesses client similarities using local model updates. FeSEM [21] combines FedSGD [12] with Expectation Maximization to group clients through a multi-center aggregation strategy, aligning each client's local model with the closest global model. Similarly, [22] introduces a clustering method based on analyzing principal angles between client data subspaces. We remark that all the mentioned methods are designed for supervised FL, assuming that client data are labeled, which is a restrictive and often unrealistic assumption. Additionally, these methods typically require broadcasting the models of all clusters to every client, demanding substantial communication bandwidth.

In the more challenging *labels-at-server* SSFL scenario, only few clustering solutions exist. FedCLAR [23] uses hierarchical clustering on clients with active learning and pseudo-labeling. However, like all approaches that rely on pseudo-labels, FedCLAR can suffer from negative learning in the presence of significant domain shifts. In addition, it also assumes to obtain few labeled data with active learning, which is not always practical. CHESSFL [24] uses hierarchical embeddings and clustering in a Hierarchical Federated Learning framework where several edge servers hold labeled data, and a central cloud server orchestrates the training process. This method is not applicable to our single-server setting where there are no gateways holding labeled data.

Finally, we mention unsupervised *Federated Clustering* methods, such as [25] and [26], which aims to cluster data, rather than models, based on a global similarity measure without sharing data. However, these solutions operate in very different settings, thus we do not consider Federated Clustering methods.

III. Problem Formulation

We consider an SSFL framework involving a *server*, denoted by $\mathcal{S}$, and a set of *clients*, $\mathcal{C}_1, \ldots, \mathcal{C}_M$, which communicate with $\mathcal{S}$ to collaboratively train models. We assume a *labels-at-server* scenario, where the server $\mathcal{S}$ holds a *labeled dataset* $\{X_\mathcal{S}, Y_\mathcal{S}\}$, consisting of input samples $X_\mathcal{S}$ and their corresponding labels $Y_\mathcal{S}$. In contrast, each client $\mathcal{C}_i$ has access only to an *unlabeled dataset* X_i, which contains input data without associated labels and is never exchanged with the server or other clients. We define $\phi_\mathcal{S} = \phi_\mathcal{S}(x, y)$ and $\phi_i = \phi_i(x, y)$ as the joint distributions of inputs and labels at the server and a given client $\mathcal{C}_i$, respectively. In the multiple domain shift settings, we consider that each client $\mathcal{C}_i$ receives data from a distribution ϕ_i that differs from $\phi_\mathcal{S}$, and at least two clients exhibit distinct distributions. Our goal is to develop a classification model for each client, despite the lack of labeled data on the client side.

IV. Proposed Solution

In this Section we present *ClusterSSFDA*, a novel solution that leverages clustering to effectively address multiple domain shifts in realistic labels-at-server SSFL settings. As in traditional FL methods, training occurs over multiple communication rounds, where the server and clients exchange updated models and parameters without sharing raw data. ClusterSSFDA tackles both client-server and client-client domain shifts by clustering clients based on the agreement of their fine-tuned models evaluated on labeled server data. The core of ClusterSSFDA lies in its clustering algorithm, which is executed at the server at the end of each communication round. Following the clustering step, the server aggregates models within each cluster and redistributes the aggregated models to the respective clients. In the following, we describe the main stages of ClusterSSFDA, as illustrated in Figure 1.

A. Framework

To illustrate the mechanism behind SSFL solutions for classification, it is convenient to divide the classifier $\mathcal{M}$ into an encoder $\mathcal{E}(\cdot;\theta)$ and a classification head $\mathcal{K}(\cdot;\omega)$, where θ and ω denote the parameters of $\mathcal{E}$ and $\mathcal{K}$, respectively. Thus, inference by $\mathcal{M}(\cdot;\theta,\omega)$ can be written as $\mathcal{K}(\mathcal{E}(\cdot;\theta);\omega)$.

ClusterSSFDA operates as follows.

Initialization.

The server initializes the model $\mathcal{M}$ by conducting end-to-end training on its labeled dataset $\{X_\mathcal{S}, Y_\mathcal{S}\}$ (Figure 1.a). The optimal parameters $\theta^\mathcal{S}$ and ω are estimated by minimizing the standard cross-entropy loss $\mathcal{L}_X$. The learned weights $\theta_\mathcal{S}$ and ω are then broadcast to all clients for inference and fine tuning.

Local Fine Tuning.

Given the lack of supervision at clients, each client updates only the encoder parameters θ using an unsupervised loss function, while keeping the classification head $\mathcal{K}$ fixed (see Figure 1.b). Specifically, we adopt the covariance alignment loss from SemiFDA [8], which minimizes the distance between the feature covariance matrices of client and server encoders, as in DeepCORAL [27], defined as:

$$\mathcal{L}_U(\theta_i, X_i, \Sigma_S) = ||\Sigma_S - \text{cov}[\mathcal{E}(X_i, \theta_i)]||_F^2 \,. \tag{1}$$

Here $\text{cov}[\cdot]$ denotes the operator computing the sample covariance, and Σ_S denotes the covariance matrix of the features extracted by $\mathcal{E}$ at server, which is exchanged between the server and clients only once after the model initialization. This process results in personalized encoder weights θ_i for each client $\mathcal{C}_i$, which are sent back to the server. The parameters ω of the classification head $\mathcal{K}$ are not updated nor sent back.

Clients Clustering

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

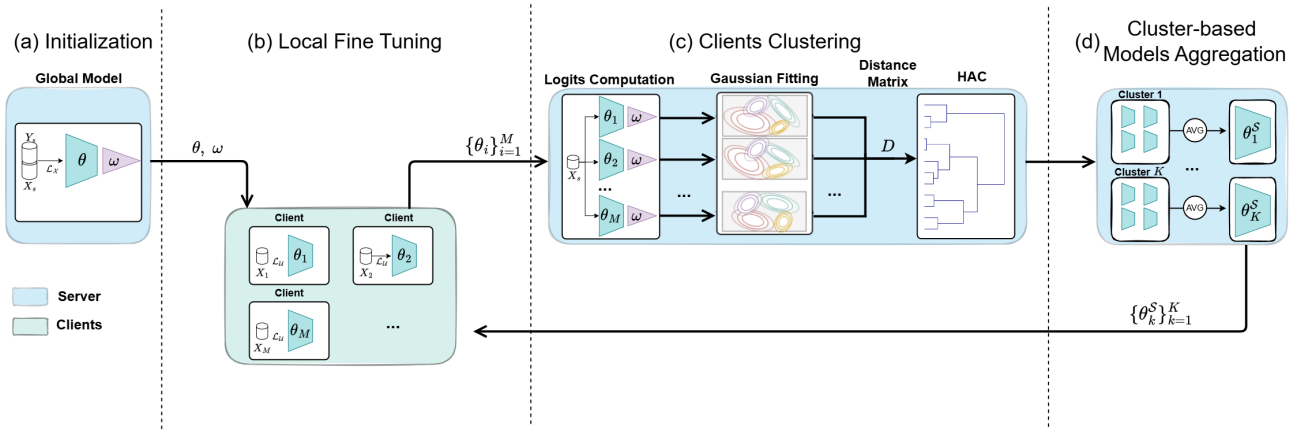


Fig. 1: Illustration of how ClusterSSFDA works. In the *initialization* step, a model $\mathcal{M}$, composed of an encoder $\mathcal{E}(\cdot, \theta_{\mathcal{S}})$ and a classification head $\mathcal{K}(\cdot, \omega)$, is trained in an end-to-end fashion using the labeled dataset $\{X_{\mathcal{S}}, Y_{\mathcal{S}}\}$. The learned parameters $\theta_{\mathcal{S}}, \omega$ are sent to the clients. Each client $\mathcal{C}_i$ performs a *local fine tuning*, obtaining a new set of encoder parameters θ_i that is sent back to the server, that collects them and perform the *clustering*. More precisely, the server combines each client-specific encoder with the shared classification head and computes the class-conditioned logits on its data $X_{\mathcal{S}}$. Then a Gaussian is fitted for each class-conditioned logits subset, and the sum of the class-conditioned symmetric Kullback-Leibler divergences is used as distance for a Hierarchical Agglomerative Clustering. Finally, the server *aggregates* the encoders of the clients that belongs to the same cluster, and send the aggregated encoders back to the corresponding clients.

After receiving the clients' encoder weights $\{\theta_i\}$, the server constructs *personalized classifiers* $\mathcal{M}_i = \mathcal{M}(\cdot; \theta_i, \omega)$ by combining each locally updated encoder $\mathcal{E}(\cdot; \theta_i)$ with the fixed classification head $\mathcal{K}(\cdot; \omega)$. Since these models combine client-specific encoders $\mathcal{E}(\cdot; \theta_i)$ and a fixed shared classification head $\mathcal{K}(\cdot; \omega)$, differences between clients are entirely determined by the encoder weights θ_i. However, as it will be discussed in Section VI, directly clustering based on encoder weights is impractical due to the curse of dimensionality, which makes the clustering process extremely challenging. For this reason, we propose to cluster clients based on the predictions of their personalized models $\mathcal{M}_1, \dots, \mathcal{M}_M$ on the labeled server data $X_{\mathcal{S}}$. In particular, since the classification head $\mathcal{K}(\cdot; \omega)$ is shared among all clients, it can be interpreted as a projection over the logits space, making logits a more compact and task-relevant representation for clustering. In neural networks, logits are the raw outputs of the last layer before applying the softmax activation function. For each client C_i and class label ℓ, we compute the *class-conditioned logits* as:

$$L_i^{\ell} = \{\overline{\mathcal{M}_i}(X_{\mathcal{S}}; \theta_i, \omega) \,|\, Y(X_{\mathcal{S}}) = \ell\}, \tag{2}$$

where $\overline{\mathcal{M}_i}$ represents the model without applying the softmax activation, and $Y(\cdot)$ denotes the class label of the server data (see Figure 1.c *Logits Computation*). Note that the predictions $P_i = \mathcal{M}_i(X_{\mathcal{S}}; \theta_i, \omega)$ are then obtained by applying the softmax function to the logits L_i, i.e., $P_i = \text{softmax}(L_i)$.

Following [28], we assume that logits follow a class-conditioned multivariate Gaussian distribution. For each class ℓ, we fit a Gaussian distribution $\hat{\psi}_i^{\ell} = \mathcal{N}(\mu_i^{\ell}, \Sigma_i^{\ell})$ to the class-conditioned logits L_i^{ℓ} (see Figure 1.c *Gaussian Fitting*). The agreement between two models $\mathcal{M}_i$ and $\mathcal{M}_j$ (and thus between clients C_i and C_j) is then measured by computing the distance between their corresponding Gaussian distributions $\hat{\psi}_i^{\ell}$ and $\hat{\psi}_j^{\ell}$ for each class ℓ. We quantify the discrepancy between these distributions using the *symmetric Kullback-Leibler* (sKL) divergence:

$$sKL(\widehat{\psi}_i^{\ell}, \widehat{\psi}_j^{\ell}) = \frac{1}{2}\left(D_{KL}(\widehat{\psi}_i^{\ell} \| \widehat{\psi}_j^{\ell}) + D_{KL}(\widehat{\psi}_j^{\ell} \| \widehat{\psi}_i^{\ell})\right), \tag{3}$$

where D_{KL} indicated the Kullback-Leibler divergence, which is not symmetric and therefore could not be used as a distance. We remark that the KL divergence D_{KL} can be computed in closed form for multivariate Gaussian distributions [29]. The total discrepancy between clients C_i and class label ℓ is computed by summing the sKL divergences across all classes.

Using these pairwise discrepancies, we construct a distance matrix $D \in \mathbb{R}^{M \times M}$, where M is the number of clients (see Figure 1.c *Distance Matrix*):

$$D_{ij} := \sum_{\ell} sKL(\widehat{\psi}_i^{\ell}, \widehat{\psi}_j^{\ell}) \quad \text{for } 1 \leq i, j \leq M. \tag{4}$$

The distance matrix D is then fed into a Hierarchical Agglomerative Clustering (HAC) [30] algorithm with complete linkage, which incrementally builds a hierarchical dendrogram (see Figure 1.c *HAC*). HAC starts by treating each client as its own cluster and repeatedly merges the closest clusters based on the distance matrix D until the predefined number of clusters is reached.

Cluster-based Models Aggregation.

The classifiers $\mathcal{M}_i$ within the same cluster k are aggregated using a traditional Federated Averaging approach, i.e. averaging their weights to obtain an aggregated classifier $\mathcal{M}_k$,

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

again composed of an encoder $\mathcal{E}(\cdot;\theta_{\mathcal{S}}^k)$ and a classification head $\mathcal{K}(\cdot;\omega_{\mathcal{S}}^k)$, with parameters $\theta_{\mathcal{S}}^k$ and $\omega_{\mathcal{S}}^k$, respectively (see Figure 1.d). Importantly, the parameters $\omega_{\mathcal{S}}^k$ of each cluster k are identical to ω, as the classification head is never updated by the clients. As a result, the aggregation process results in K distinct classifiers on the server, each corresponding to a specific cluster. Finally, the next communication round begins by initializing the model of each client C_i in cluster k with the parameters $\theta_{\mathcal{S}}^k$ and $\omega_{\mathcal{S}}^k$ from the aggregated cluster-specific classifier $\mathcal{M}_k$.

V. Experiments

We take inspiration from the experimental framework proposed in [8], which enables the simulation of varying levels of domain shift in a federated learning context. Specifically, we focus on the most challenging scenario, characterized by both server-client and client-client domain shifts, where data distribution changes not only between the server and clients but also among the clients themselves.

To facilitate reproducibility, the code used to implement our method and conduct the experiments is publicly available for download at https://tinyurl.com/366wywwd.

A. Datasets

We evaluate our solution on two use cases: HAR and Digit Classification. In HAR, we utilize 3 public datasets: USC-HAD [2], RealWorld [1], and MobiAct [3]. These datasets, collected using tri-axial MEMS accelerometers, include recordings from diverse users (14 in USC-HAD, 15 in RealWorld, and 60 in MobiAct) with varying genders, ages, weights, and heights. Additionally, they exhibit significant differences in sampling frequency, sensor types, and performed activities, leading to severe domain shifts. Following [8], we resample all data to a frequency of 50Hz and focus on four common activities across datasets: *standing*, *walking*, *running*, and *jumping*. Despite this preprocessing, substantial differences remain among the datasets. We segment the recordings into windows of 64 samples, ensuring each window spans at least one second. These windows are referred to as samples.

In Digit Classification, we consider multiple benchmark datasets: MNIST [31], MNIST-M [32], USPS [33], ARDIS IV [34] and Synthetic Digits [35]. Each client is assigned images from a single dataset, consistent with common practices in Federated Learning literature. All digit images are resized to a spatial resolution of 32×32 pixels to ensure consistency across datasets.

B. Experimental framework

We simulate a very challenging setting affected by both server-client and client-client domain shifts. For HAR, we simulate different clients by selecting samples from individual users, and we split user data into different communication rounds, ensuring all four activities are represented in each round. We remark that HAR inherently involves multiple levels of domain shift due to variations in sensor configurations, sensor locations, and user characteristics. To create a realistic setting, we assign one dataset entirely to the server and distribute data from the other two datasets among the clients as just explained.

In Digit Classification, we consider two sub-tasks: *Rotated-MNIST (Rot-MNIST)* and *Cross-dataset Images*. In both cases, we simulate multiple domain shifts by artificially distributing data between the server and clients, following the approach in [8]. In particular, for the Rot-MNIST experiments, the server uses the original MNIST dataset (as the most standard among digits dataset), assuming to have also the label for each image. Clients data are then generated by rotating MNIST images to simulate different domains. We evaluate three configurations:

- 3Rot-MNIST: rotations of ± 45 and 180, with 30 clients (10 per rotation).
- 5Rot-MNIST: rotations of ± 45, ± 90 and 180, with 25 clients (5 per rotation).
- 7Rot-MNIST: rotations of ± 45, ± 90, ± 135 and 180, with 35 clients (5 per rotation).

For the Cross-dataset Images task, we use five Digit Classification datasets: MNIST, USPS and ARDIS-IV, which contain grayscale images, and MNIST-M and Synthetic Digits (SYN), which contain RGB images. Again, we adopt one dataset at a time on the server and we distribute data from the other four datasets among the clients, so that each client gather data belonging to a single dataset. Specifically, we simulate 10 clients per dataset, resulting in a total of 40 clients.

As the figure of merit, we consider the *average accuracy* on the clients' test sets at the conclusion of the final communication round. Specifically, we perform a total of 10 communication rounds, starting from a classification model trained on labeled server data. For simplicity, we set the participation rate to 100%. In all our experiments, we perform 20 independent runs and report the average accuracy and the standard deviations for all the considered methods.

C. Model architecture and training

As described in Section IV, ClusterSSFDA iteratively updates a classification model $\mathcal{M}$ over multiple communication rounds. Following the approach in [8], we use a standard CNN classifier for $\mathcal{M}$. This choice is motivated by the portability and efficiency of CNNs, which are well-suited for resource-constrained FL clients that require lightweight yet effective models. Notably, our CNN achieves above 90% accuracy on the server test set (unaffected by domain shift) for all the considered tasks, demonstrating that it is sufficiently robust and not overly simple.

For the HAR task, the model $\mathcal{M}$ consists of an encoder $\mathcal{E}$ and a classification head $\mathcal{K}$. The encoder $\mathcal{E}$ includes 4 1D convolutional layers with ReLu activations, each followed by a max pooling layer, and a final dense layer. The classification head $\mathcal{K}$ consists of 2 dense layers with ReLU activation, each followed by dropout, and a final dense layer with softmax activation. The total number of parameters of the HAR model is approximately $4.5k$. For the Digit Classification task, we adapt the model architecture to process image data by replacing 1D convolutional layers with 2D convolutional layers and adding

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I: HAR experiment: we iterate all the combinations using one dataset at server, and the remaining 2 at clients. For each combination, we report the accuracy of ClusterSSFDA (K=4 clusters), the single-model competitors and the weights-similarity clustering alternative. We also report the accuracy of the the supervised oracle FedAvg along with its Cluster counterpart.

Dataset at server	Centralized	AE-SSFL	SemiFL	SemiFDA	Weights similarity	ClusterSSFDA	FedAvg	
							Global	Cluster
USCHAD	.34 (.02)	.27 (.01)	.36 (.04)	.51 (.04)	**.52 (.03)**	**.52 (.03)**	.83 (.02)	.89 (.01)
RealWorld	.46 (.03)	.57 (.04)	.49 (.03)	.58 (.04)	**.67 (.04)**	.64 (.03)	.85 (.01)	.92 (.01)
MobiAct	.51 (.02)	.67 (.03)	.51 (.02)	.54 (.02)	.77 (.02)	**.78 (.02)**	.84 (.02)	.93 (.01)
Average	.44 (.02)	.50 (.03)	.45 (.03)	.54 (.03)	**.65 (.03)**	**.65 (.03)**	.84 (.02)	.91 (.01)

TABLE II: Rotated-MNIST experiment: we select the original (non-rotated) MNIST at server, and the different rotations for the clients. For each combination, we report the accuracy of ClusterSSFDA (#clusters = #rotations), the single-model competitors and the weights-similarity clustering alternative. We also report the accuracy of the the supervised oracle FedAvg along with its Cluster counterpart.

Rotation at clients	Centralized	AE-SSFL	SemiFL	SemiFDA	Weights similarity	ClusterSSFDA	FedAvg	
							Global	Cluster
3Rot-MNIST	.47 (.01)	.31 (.01)	.51 (.02)	.52 (.01)	.58 (.02)	**.69 (.01)**	.94 (.01)	.96 (.01)
5Rot-MNIST	.38 (.01)	.27 (.01)	.42 (.01)	.39 (.01)	.46 (.01)	**.53 (.01)**	.87 (.01)	.97 (.01)
7Rot-MNIST	.32 (.01)	.22 (.01)	.34 (.01)	.30 (.01)	.33 (.01)	**.41 (.01)**	.80 (.01)	.93 (.01)
Average	.39 (.01)	.27 (.01)	.42 (.01)	.40 (.01)	.46 (.01)	**.54 (.01)**	.87 (.01)	.95 (.01)

TABLE III: Cross-dataset Image experiment: we iterate all the combinations using one dataset at server, and the remaining 4 at clients. For each combination, we report the accuracy of ClusterSSFDA (K=4 clusters), the single-model competitors and the weights-similarity clustering alternative. We also report the accuracy of the the supervised oracle FedAvg along with its Cluster counterpart.

Dataset at server	Centralized	AE-SSFL	SemiFL	SemiFDA	Weights similarity	ClusterSSFDA	FedAvg	
							Global	Cluster
MNIST	.50 (.01)	.32 (.01)	.48 (.01)	.56 (.01)	**.61 (.01)**	.60 (.01)	.77 (.01)	.73 (.01)
MNIST-M	.62 (.01)	.35 (.01)	.60 (.02)	.68 (.01)	**.69 (.01)**	.68 (.01)	.83 (.01)	.82 (.01)
USPS	.39 (.01)	.24 (.01)	.48 (.01)	.45 (.01)	**.50 (.01)**	**.50 (.01)**	.76 (.01)	.75 (.01)
ARDIS IV	.43 (.01)	.29 (.01)	**.51 (.01)**	.46 (.01)	.47 (.01)	.47 (.01)	.77 (.01)	.75 (.01)
SYN	.47 (.01)	.16 (.01)	.48 (.02)	**.67 (.01)**	.66 (.01)	**.67 (.01)**	.91 (.01)	.88 (.01)
Average	.48 (.01)	.27 (.01)	.51 (.01)	.56 (.01)	**.59 (.01)**	**.59 (.01)**	.81 (.01)	.79 (.01)

batch normalization after each convolutional block. The total number of parameters for this model is approximately $330k$.

In the initialization phase (Figure 1a), we train on the server the classification model until convergence adopting an early stopping callback on the validation set (validation split = 0.2). Regarding the local fine tuning, we split each clients' data in Training and Test sets, using the mainstream 80%-20% split. The local fine-tuning consists of 30 epochs, setting the batch size to 64. For both server and clients' training we adopt Adam optimizer with a learning rate of 1e-3.

The experiments have been conducted on an NVIDIA RTX 6000 Ada Generation GPU. The runtime of ClusterSSFDA varies depending on the specific task and experiment but remains consistently under one hour.

D. Considered Methods

As alternative baselines, we consider a centralized model trained on labeled data at server (X_S, Y_S), referred to simply as *Centralized*. A fundamental requirement for any FL and SSFL scheme is in fact to outperform this pretrained server model through iterative fine-tuning on the clients. Additionally, we compare against *FedAvg* [36], the most popular supervised FL scheme, which serves as an oracle baseline since it utilizes labeled data on the clients.

As SSFL competitors, we firstly consider *SemiFDA* [8], which has been demonstrated to effectively handle domain shift between the server and the clients. We also compare our results against *AE-SSFL* [7], which is based on AutoEncoders, and against *SemiFL* [6], a state-of-the-art SSFL method based on pseudo-labeling.

Furthermore, we adapt a traditional Clustered Federated Learning algorithm based on weights similarity, which was originally designed for supervised FL, to the more challenging SSFL setting. Traditional federated clustering algorithms typically cluster clients using the weight similarity of their classification heads [20]. However, in popular SSFL solutions, the classification head is shared across all clients and cannot be used for clustering. To address this, we modify the existing supervised solution of [20] to make it suitable for a SSFL setting. In particular, we cluster clients based on the weights

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

similarity of their client-specific encoders, which are fine-tuned locally in an unsupervised manner. It is important to note that this clustering approach, denoted as *Weights Similarity* in our experiments, has not been previously introduced in SSFL and represents an alternative rather than a direct competitor.

To further demonstrate the flexibility of ClusterSSFDA, we extend FedAvg by incorporating the same clustering algorithm, referring to this approach as *Cluster-FedAvg*. In Cluster-FedAvg, the clustering algorithm from ClusterSSFDA is applied to the client-specific encoders $\mathcal{E}_i$ concatenated to the client-specific classification heads $\mathcal{K}_i$ which are locally updated in a supervised manner, as is standard in FedAvg. In practice, we apply ClusterSSFDA on the logits computed on the client-specific model $\overline{\mathcal{M}}(\cdot; \theta_i, \omega_i)$, which consists in the values before softmax for the model fine-tuned at clients. It is important to note that, like FedAvg, Cluster-FedAvg assumes the availability of labeled data on the client side. As such, Cluster-FedAvg is not considered a direct competitor to our method, but represents an oracle supervised solution.

E. Results

In HAR experiments, we evaluate all three possible dataset combinations by assigning one dataset entirely to the server and distributing the remaining two among the clients. In Table I, we report the results achieved by ClusterSSFDA compared to SSFL competitors that perform a global aggregation strategy (SemiFDA, AE-SSFL, and SemiFL), as well as the weights-similarity clustering alternative (Weights-Similarity). Additionally, we include the results of the centralized baseline, along with FedAvg and the Cluster-FedAvg solutions, which serve as upper bounds, since they rely on labeled data at the client level. The Table reports both the average performance across the three experiments and the detailed results for each specific dataset combination.

The results demonstrate that integrating clustering into a traditional SSFL framework is beneficial, confirming that aggregating all clients into a single global model is suboptimal in the presence of domain shift. Specifically, ClusterSSFDA consistently outperforms its underlying scheme, SemiFDA, and all other SSFL competitors that train a single model across all scenarios. Furthermore, the Weights-Similarity clustering alternative also proves effective, achieving results comparable to ClusterSSFDA. This is likely because, in HAR, the dimensionality of the encoder weights is not excessively high (4.5k, as detailed in Section IV.C), making similarity-based comparisons more tractable.

Moreover, Table I also highlights that ClusterSSFDA is effective in addressing domain shifts even when applied to the supervised oracle FedAvg, as its clustered variant consistently outperforms the global version across all dataset combinations. In contrast, as shown in [8], AE-SSFL and SemiFL are ineffective in handling domain shift. Consequently, we do not integrate our clustering algorithm with these methods, as it would not provide meaningful improvements within such SSFL frameworks.

Tables II and III present the results of the Digit Classification experiments. In particular, in Table II we report the results of the *Rotated-MNIST* task, while in Table III we report the ones for *Cross-dataset Images*.

In the *Rotated-MNIST* task, ClusterSSFDA demonstrates a significant performance advantage over all baselines and competitors. On average, it achieves a 14% improvement compared to SemiFDA and an 8% improvement over the Weights-Similarity clustering alternative. The effectiveness of our clustering algorithm is further validated by Cluster-FedAvg, which outperforms its non-clustered counterpart by 8% and confirm that aggregating all clients into a single global model is suboptimal. Additionally, while the Weights-Similarity approach shows benefits over traditional global aggregation, its performance remains inferior to ClusterSSFDA. This discrepancy is likely due to challenges associated with the curse of dimensionality in image-based tasks, which can limit the effectiveness of the weights-similarity method.

In the *Cross-dataset Images* experiment, we observe that ClusterSSFDA once again outperforms SemiFDA and all the other single-model competitors. However, when comparing Table II against the MNIST row of Table III, it emerges that the domain shift introduced by different datasets (77% accuracy for FedAvg Global) is more substantial than the one introduced by rotations (87% average accuracy for FedAvg Global in Table II), and that it is difficult to learn features generalizing at best among all the domains. Additionally, the clustering in the supervised settings is not beneficial, probably failing at gathering together clients from the same domain and just learning sort of domain invariant features but from a smaller number of clients, as the inferior performance of ClusterFedAvg attest (73% accuracy in the MNIST row of Table III). These settings are very challenging for clustering, as confirmed by the semi-supervised experiments, where ClusterSSFDA grants only a marginal improvement (3% on average) compared to its single-model counterpart SemiFDA. Probably, the clustering features learned in a semi-supervised settings suffer of the same limitations discussed above for the supervised case. The modest performance improvement is however common to both clustering based on Weights-Similarity and ClusterSSFDA.

Overall, our results confirm that aggregating all clients into a single global model is suboptimal in the presence of domain shift among clients. Our clustering approach, however, consistently delivers improved classification performance in such scenarios.

VI. Discussion

In this Section, we discuss key design decisions behind our proposed solution and its adaptability to various scenarios.

A. Clustering scheme

As mentioned in Section V-D, traditional Clustered Federated Learning methods that cluster clients based on the similarity of weights of their classification heads [20] are not suitable for labels-at-server SSFL, since the classification head is shared (and fixed in SemiFDA) across all clients. In our

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Accuracies of ClusterSSFDA and Cluster-FedAvg in the Rotated MNIST experiments varying the number of clusters K.

Dataset at server	ClusterSSFDA K=3	ClusterSSFDA K=5	ClusterSSFDA K=7	ClusterSSFDA K=9	Cluster-FedAvg K=3	Cluster-FedAvg K=5	Cluster-FedAvg K=7	Cluster-FedAvg K=9
3Rot-MNIST	.69 (.01)	.68 (.01)	.67 (.01)	.67 (.01)	.96 (.01)	.97 (.01)	.97 (.01)	.97 (.01)
5Rot-MNIST	.51 (.01)	.53 (.01)	.54 (.01)	.52 (.01)	.96 (.01)	.97 (.01)	.97 (.01)	.97 (.01)
7Rot-MNIST	.37 (.01)	.39 (.01)	.41 (.01)	.41 (.01)	.89 (.01)	.93 (.01)	.93 (.01)	.94 (.01)
Avg	.52 (.01)	.53 (.01)	.54 (.01)	.53 (.01)	.94 (.01)	.96 (.01)	.96 (.01)	.96 (.01)

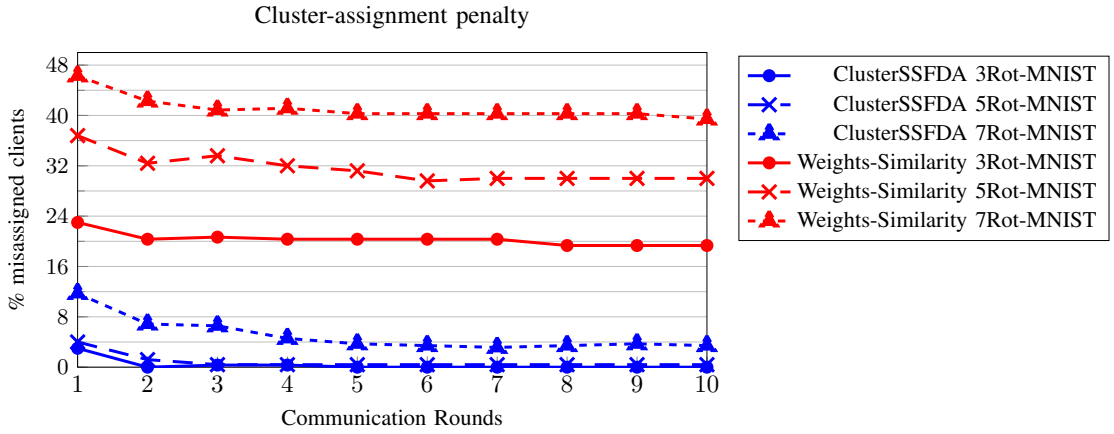


Fig. 2: Cluster Assignment Penalty achieved by ClusterSSFDA and Weights-similarity clustering on the three Rotated-MNIST datasets. The plots represent the median values computed over 20 runs. ClusterSSFDA consistently achieves a lower penalty compared to weights-similarity, indicating more accurate clustering of clients. This aligns with the superior accuracy achieved by ClusterSSFDA, as reported in Table II.

experiments, we present an alternative approach that enables clustering in SSFL scenarios by assessing the similarity of client encoder weights only. However, this approach could be impractical, particularly for image-based tasks, due to the high dimensionality of encoder weights, which makes similarity-based comparisons intractable (i.e., curse of dimensionality). This limitation is demonstrated by our Rotated-MNIST experiments, where weights-similarity clustering yields inferior performance. Therefore, we base our clustering algorithm on the agreement among the models evaluated on the server data, as detailed in Section IV. Specifically, we utilize the class-conditioned logits L_i^ℓ output by each model $\overline{\mathcal{M}}_i$, instead of applying the softmax activation function. This choice avoids the potential loss of representative information caused by softmax, as demonstrated in [37], where logits have been shown to be more effective in identifying out-of-distribution samples. In our *labels-at-server* scenario, we assume that a representative labeled dataset is available at the server to enable the correct computation of class-conditioned logits. Such datasets are not unrealistic in practice, as they can often be obtained from publicly available sources. Finally, we remark that ClusterSSFDA is a rather general SSFL solution that is potentially adaptable to tasks other than classification, such as regression. In fact, we can define the models agreement for any deep learning model fine tuned at clients, by feeding this model with supervised data at the server and fitting class-conditioned Gaussian distributions on the output. As discussed in Section IV-A, in ClusterSSFDA we assume that logits follow a class-conditioned multivariate Gaussian distribution, in line with [28]. We acknowledge, however, that this assumption may not always hold in practical scenarios, and its violation could lead to suboptimal performance. Therefore, to overcome this limitation, it would be preferable in certain cases to adopt more flexible distributional assumptions that can better capture complex or non-Gaussian structures in the data.

B. Robustness Against Number of Clusters

The number of clusters is a crucial hyperparameter in many clustering methods. In this Section, we analyze the robustness of ClusterSSFDA with respect to a varying number of clusters in the Digit Classification scenario, specifically for the Rotated-MNIST experiments. Additionally, we evaluate also the supervised baseline FedAvg while incorporating our clustering algorithm (Cluster-FedAvg).

We run our clustering algorithm with $K = \{3, 5, 7, 9\}$ for each of the three subtasks *3Rot-MNIST, 5Rot-MNIST* and *7Rot-MNIST*. Table IV reports the accuracies achieved varying the number of clusters in the analysed tasks. The results indicate that the average accuracies of ClusterSSFDA remain nearly constant across different cluster configurations K in all three sub-tasks. In particular, from our results, we observe that

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

is preferable to overestimate rather than underestimate the number of clusters. This is reasonable, as we are dealing with a challenging scenario affected by domain shift (specifically rotations) between the clients: underestimating the number of clusters risks grouping clients with different data rotations together, potentially degrading the quality of the learned representations and overall model performance.

To better understand the clustering results, we analyze the Rotated-MNIST task by comparing the obtained clusters to the *ground-truth clusters*, defined by the dataset (i.e. specific rotation) where client data are from. While this labeling is not reliable in HAR due to client heterogeneity, in the Rotated-MNIST experiments there is no domain shift between clients from the same dataset. To evaluate clustering quality, we introduce a custom metric called the *cluster-assignment penalty*. For each cluster, we identify the most frequent dataset among clients and treat it as the true assignment. The remaining clients in that cluster are considered misassigned, and the cluster-assignment penalty is defined as the percentage of misassigned clients. We consider a clustering to be optimal when each cluster contains clients from a single dataset, yielding a cluster-assignment penalty of zero.

We compute cluster-assignment penalty for both ClusterSSFDA and Weights-Similarity clustering for all the three Rot-MNIST sub-tasks. We plot the mean values in Figure 2. For ClusterSSFDA, the cluster-assignment penalty decreases with the communication rounds, tending to 0, i.e., clusters from a single dataset, for each of the three considered tasks. Analogously, for Weights-similarity the cluster-assignment penalty decreases along the communication rounds. However, the penalty is significantly higher with respect to ClusterSSFDA, reaching in the best case a penalty of 20% for the 3Rot-MNIST task. These results are consistent with those reported in Table II, proving that the superior performance of ClusterSSFDA with respect to Weights-Similarity clustering is related to a more accurate cluster assignments. We emphasize that, although the choice of the optimal number of clusters remains a crucial hyperparameter in our setting and deserves further investigation, our results demonstrate that overestimating this parameter is a safer strategy. This approach minimizes the risk of grouping highly dissimilar clients together, which would otherwise degrade the performance of our framework.

C. Iterative vs. Single Clustering

ClusterSSFDA relies on an iterative clustering scheme, which is executed at the end of each communication round. This choice aligns well with the inherent nature of federated learning settings, where clients continuously collect data. To prove the beneficial effect of an iterative clustering approach, we compare our results on the 5Rot-MNIST task against a *single-clustering* approach. In particular, we try both performing the clustering after the first communication round (denoted as *first-clustering*) and also performing a delayed clustering after 5 communication rounds of global aggregation (denoted as *delayed-clustering)*. In Table V we compare the results of the different approaches. We notice that performing an iterative clustering is the best choice for ClusterSSFDA, outperforming first-clustering and delayed-clustering by 3% and 8%, respectively. Additionally, we notice that delayed-clustering is the worst-performing approach for both ClusterSSFDA and Cluster-FedAvg: due to the domain shift between the clients, performing global aggregation along the first 5 communication rounds results in performance degradation, which can be only partially compensated by a delayed clustering.

TABLE V: Accuracies on 5Rot-MNIST task of ClusterSSFDA and Cluster-FedAvg with the original *iterative clustering*, against two single-clustering alternatives, i.e. *first-clustering* and *delayed-clustering*. For *first-clustering*, we perform the clustering algorithm after the first communication round, while for the *delayed-clustering* we perform it after 5 rounds of global aggregation.

	ClusterSSFDA	Cluster-FedAvg
iterative-clustering	**.53 (.01)**	**.97 (.01)**
first-clustering	.50 (.01)	**.97 (.01)**
delayed-clustering	.45 (.01)	.96 (.01)

As expected, there is a trade-off between the computational load of iterative clustering and the performance of the aggregated models. However, we remark that the computational overhead of an iterative clustering is negligible in our setting, as all the operations related to the clustering are computed on the server, which is assumed to have potentially unlimited computational resources.

D. Scalability and Privacy of ClusterSSFDA

As discussed in Section IV-A, the KL divergence D_{KL} admits a closed-form expression when computed between multivariate Gaussian distributions [29], which makes the distance evaluation computationally efficient. Additionally, the adopted hierarchical clustering procedure has complexity scaling with N^2, since pairwise distances among all N clients must be computed to construct the distance matrix. While alternative clustering schemes such as k-means exhibit a lower complexity of $\mathcal{O}(KN)$, this computational overhead does not pose a significant limitation in our framework, as all clustering operations are executed on the server side, where sufficient computational resources are assumed to be available.

Finally, we discuss the privacy implications of ClusterSSFDA and its model-agreement clustering scheme. As outlined in Section IV-A, in line with traditional FL approaches, our framework does not require clients to share any additional information beyond the standard model parameters. In fact, the clustering is performed entirely at the server by running inference on server data and computing the class-conditioned logits, without exposing further client-specific data.

VII. Conclusion

In this paper, we introduce ClusterSSFDA, the first SSFL solution that adopts clustering to effectively address domain shift. ClusterSSFDA employs a novel clustering algorithm specifically designed for labels-at-server SSFL, which assesses

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.

on models' agreement to group similar clients and update multiple cluster-specific models at server.

We assess the performance of ClusterSSFDA on HAR, a popular SSFL task which is inherently affected by multiple levels of domain shifts, and two tasks related to Digit Classification. Our experiments demonstrate that ClusterSSFDA outperforms existing SSFL solutions in challenging scenarios affected by multiple domain shifts. Moreover, we show that ClusterSSFDA is flexible and can be effectively combined with supervised FL frameworks, improving their performance. Finally, we demonstrate the robustness of our solution when varying the number of clusters, a crucial hyperparameters in clustering algorithms.

Future work concerns the design of new client discrepancies measures to improve clustering as well as investigating heuristic principles to estimate the optimal number of clusters.

REFERENCES

[1] T. Sztyler and H. Stuckenschmidt, "On-body localization of wearable devices: An investigation of position-aware activity recognition," in *PerCom*, 2016.

[2] M. Zhang and A. Sawchuk, "Usc-had: a daily activity dataset for ubiquitous activity recognition using wearable sensors," 2012.

[3] G. Vavoulas, C. Chatzaki, T. Malliotakis, M. Pediaditis, and M. Tsiknakis, "The mobiact dataset: Recognition of activities of daily living using smartphones," 2016.

[4] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, *Dataset shift in machine learning*. Mit Press, 2022.

[5] W. Jeong, J. Yoon, E. Yang, and S. J. Hwang, "Federated semi-supervised learning with inter-client consistency & disjoint learning," *arXiv preprint arXiv:2006.12097*, 2020.

[6] E. Diao, J. Ding, and V. Tarokh, "Semifl: Semi-supervised federated learning for unlabeled clients with alternate training," in *NeurIPS*, vol. 35, 2022.

[7] Y. Zhao, H. Liu, H. Li, P. Barnaghi, and H. Haddadi, "Semi-supervised federated learning for activity recognition," *arXiv preprint arXiv:2011.00851*, 2020.

[8] M. Craighero, G. Rossi, B. Rossi, D. Carrera, D. Stucchi, P. Fragneto, and G. Boracchi, "Semifda: Domain adaptation in semi-supervised federated learning," in *IEEE International Conference on Data Mining (ICDM)*. IEEE, 2024.

[9] X. Yang, Y. Gu, K. Wei, and C. Deng, "Exploring safety supervision for continual test-time domain adaptation," in *IJCAI*, 2023.

[10] D. Berthelot, N. Carlini, I. Goodfellow, N. Papernot, A. Oliver, and C. A. Raffel, "Mixmatch: A holistic approach to semi-supervised learning," *NeurIPS*, vol. 32, 2019.

[11] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "Fixmatch: Simplifying semi-supervised learning with consistency and confidence," *NeurIPS*, vol. 33, 2020.

[12] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, 2017.

[13] S. Ek, R. Rombourg, F. Portet, and P. Lalanda, "Federated self-supervised learning in heterogeneous settings: Limits of a baseline approach on har," in *PerCom Workshops*. IEEE, 2022.

[14] M. Das, Z. Liu, X. Chen, X. Yuan, and L. Zhang, *Embracing Semi-supervised Domain Adaptation for Federated Knowledge Transfer*, 02 2024, pp. 100–113.

[15] H. Lin, J. Lou, L. Xiong, and C. Shahabi, "Semifed: Semi-supervised federated learning with consistency and pseudo-labeling," 08 2021.

[16] A. T. Nguyen, P. Torr, and S. N. Lim, "Fedsr: A simple and effective domain generalization method for federated learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 38 831–38 843, 2022.

[17] R. Zhang, Q. Xu, J. Yao, Y. Zhang, Q. Tian, and Y. Wang, "Federated domain generalization with generalization adjustment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 3954–3963.

[18] F. Sattler, K.-R. Müller, and W. Samek, "Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 8, pp. 3710–3722, 2020.

[19] A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An efficient framework for clustered federated learning," *Advances in neural information processing systems*, vol. 33, pp. 19 586–19 597, 2020.

[20] C. Briggs, Z. Fan, and P. Andras, "Federated learning with hierarchical clustering of local updates to improve training on non-iid data," in *2020 international joint conference on neural networks (IJCNN)*. IEEE, 2020, pp. 1–9.

[21] G. Long, M. Xie, T. Shen, T. Zhou, X. Wang, and J. Jiang, "Multi-center federated learning: clients clustering for better personalization," *World Wide Web*, vol. 26, no. 1, pp. 481–500, 2023.

[22] S. Vahidian, M. Morafah, W. Wang, V. Kungurtsev, C. Chen, M. Shah, and B. Lin, "Efficient distribution similarity identification in clustered federated learning via principal angles between client data subspaces," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 8, 2023, pp. 10 043–10 052.

[23] R. Presotto, G. Civitarese, and C. Bettini, "Fedclar: Federated clustering for personalized sensor-based human activity recognition," in *2022 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, 2022, pp. 227–236.

[24] A.-J. Farcas, M. Lee, A. Payani, H. Latapie, R. R. Kompella, and R. Marculescu, "Chessfl: Clustering hierarchical embeddings for semi-supervised federated learning," in *2024 IEEE/ACM Ninth International Conference on Internet-of-Things Design and Implementation (IoTDI)*. IEEE, 2024, pp. 122–133.

[25] E. S. Lubana, C. I. Tang, F. Kawsar, R. P. Dick, and A. Mathur, "Orchestra: Unsupervised federated learning via globally consistent clustering," *arXiv preprint arXiv:2205.11506*, 2022.

[26] J. Chung, K. Lee, and K. Ramchandran, "Federated unsupervised clustering with generative models," in *AAAI 2022 international workshop on trustable, verifiable and auditable federated learning*, vol. 4, 2022.

[27] B. Sun and K. Saenko, "Deep coral: Correlation alignment for deep domain adaptation," in *Computer Vision–ECCV Workshops*, 2016.

[28] N. Ding, Y. Xu, Y. Tang, C. Xu, Y. Wang, and D. Tao, "Source-free domain adaptation via distribution estimation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 7212–7222.

[29] J. R. Hershey and P. A. Olsen, "Approximating the kullback leibler divergence between gaussian mixture models," in *2007 IEEE International Conference on Acoustics, Speech and Signal Processing - ICASSP '07*, vol. 4, 2007, pp. IV–317–IV–320.

[30] C. C. J. Bridges, "Hierarchical cluster analysis," *Psychological Reports*, vol. 18, pp. 851 – 854, 1966. [Online]. Available: https://api.semanticscholar.org/CorpusID:144961859

[31] L. Deng, "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.

[32] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *International conference on machine learning*. PMLR, 2015, pp. 1180–1189.

[33] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.

[34] H. Kusetogullari, A. Yavariabdi, A. Cheddad, H. Grahn, and J. Hall, "Ardis: a swedish historical handwritten digit dataset," *Neural Computing and Applications*, vol. 32, no. 21, pp. 16 505–16 518, 2020.

[35] P. Roy, S. Ghosh, S. Bhattacharya, and U. Pal, "Effects of degradations on deep neural network architectures," *arXiv preprint arXiv:1807.10108*, 2018.

[36] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, 2017.

[37] S. Vaze, K. Han, A. Vedaldi, and A. Zisserman, "Open-set recognition: A good closed-set classifier is all you need," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=5hLP5JY9S2d

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 06,2026 at 14:09:20 UTC from IEEE Xplore. Restrictions apply.